EigenSkill-Q: Verifiable Policy Bypass for Sensitivity-Rate-Distortion Guided Mixed-Precision LLM Quantization

Short-cycle manuscript draft.
Date: 2026-06-04
Status: research draft with verified policy-kernel artifacts, not yet a full submission-ready quantization benchmark paper.

Abstract

Mixed-precision post-training quantization is one of the most practical routes to deploying large language models under memory and latency constraints. Existing methods usually treat bit allocation, rotation, outlier handling, residual compensation, and KV-cache compression as algorithmic choices implemented inside offline quantization pipelines. In contrast, we study a hybrid control-plane view: low-entropy quantization-policy decisions are routed away from token-level language generation and executed by deterministic, verifiable policy kernels.

We propose **EigenSkill-Q**, a sensitivity-rate-distortion policy-bypass framework for edge-oriented LLM serving. The framework models quantization control as a constrained decision problem over bit-width assignment, rotation or preconditioning, residual correction, outlier protection, and KV-cache format selection. A lightweight semantic router may identify the policy skill, but the final numeric decision is computed by a deterministic kernel. This separates semantic triggering from safety-critical numeric execution.

As an initial artifact, we construct a no-leak synthetic quantization-policy dataset with five skills and implement both a strict Python oracle and a standalone C++ policy evaluator. The Python oracle reaches 100% full JSON accuracy on eval/test. The C++ evaluator reaches 100% decision-field accuracy on 400 eval and 400 test samples, at approximately 8261 and 6924 rows/s on local Windows/MSVC. A short SmolLM2-360M LoRA smoke run fails to learn the same numeric rules, giving negative evidence that small SFT alone is a poor executor for deterministic quantization policies. The current result does not yet establish superiority over GPTQ, AWQ, SmoothQuant, QuaRot, SpinQuant, QuIP, AQLM, or KV-cache quantization baselines. It establishes a narrower but useful systems and learning question: when quantization control is structured, verifiable, and low-entropy, should it be generated by the LLM, learned as a policy, or bypassed as a deterministic kernel?

1. Introduction

LLM quantization has moved from a simple storage-compression trick to a central algorithm-system co-design problem. Modern post-training quantiza-

tion methods must decide which layers are sensitive, which activations contain outliers, which rotations reduce quantization difficulty, which residual errors deserve low-rank compensation, and how aggressively KV caches can be compressed without destroying long-context quality.

These decisions are not pure natural-language tasks. They are constrained numeric policies. A policy such as "protect top channels when outlier risk exceeds a threshold" or "use int4 token-wise KV cache when context is long and sensitivity is low" is closer to a control rule than to open-ended generation. Yet in many LLM-agent or edge-AI prototypes, the entire decision is delegated to a small model, which must emit JSON correctly, preserve schema, and compute numeric thresholds. This is fragile, slow, and hard to verify.

EigenSkill-Q starts from a different premise:

```
Let the model route and explain; let deterministic kernels execute numeric policy.
```

This paper draft develops that idea for mixed-precision LLM quantization. The core contribution is not a new weight-only quantizer by itself. The proposed contribution is a **policy-bypass layer** that sits above quantization algorithms and below natural-language orchestration. It treats quantization control as a constrained decision problem and exposes policy skills that can be implemented either as deterministic kernels, learned contextual bandits, or constrained reinforcement learning policies.

The current short-cycle artifact verifies the deterministic-kernel corner of this design. It does not yet claim full LLM compression quality.

2. Related Work

Weight And Activation Quantization

GPTQ introduced a practical second-order post-training quantization procedure for generative transformers. AWQ showed that activation-aware protection of a small fraction of salient weights can preserve quality under low-bit weight quantization. SmoothQuant moved activation outliers into weights through a mathematically simple rescaling transformation, enabling more efficient weight-activation quantization.

These methods motivate EigenSkill-Q's sensitivity features: Hessian or Fisher proxies, activation magnitude, per-channel outlier scores, and layer-wise distortion estimates.

Rotation And Incoherence

Recent methods such as QuaRot, SpinQuant, QuIP, and related incoherence-based approaches use rotations, Hadamard transforms, learned orthogonal transforms, or trellis/product-code ideas to make quantization error more uniform. This line of work suggests that "rotation choice" is itself a policy variable, not a fixed preprocessing step.

EigenSkill-Q treats rotation selection as one of the policy skills:

```
rotation_select: (block imbalance, kurtosis, target format) -> rotation policy
```

Residual And Vector Quantization

OmniQuant and AQLM-style methods demonstrate that calibration-time optimization, additive codebooks, and residual compensation can improve low-bit quality. In EigenSkill-Q, this motivates a residual-patch skill:

```
residual_patch: spectrum/error energy -> low-rank residual rank
```

KV-Cache Quantization

Long-context serving makes KV-cache memory a dominant bottleneck. KIVI, KVQuant, and mixed KV-cache quantization methods show that key/value tensors can often be compressed with asymmetric or mixed precision rules. EigenSkill-Q therefore includes:

```
kv_policy: (context length, key sensitivity, value sensitivity) -> KV policy
```

Skill Routing And Tool Use

Tool-calling LLM systems already separate semantic recognition from external execution. EigenSkill-Q applies this principle to quantization policy. The key difference is that the tool is not a web API or calculator; it is a local micro-policy kernel that controls how model tensors are compressed and served.

3. Problem Formulation

Let a model contain layers indexed by `l = 1, ..., L`. For each layer, let `W_l` be a weight matrix and let `A_l` denote calibration activations. A quantization policy chooses:

- bit-width or numeric format `b_l`;
- rotation/preconditioner `R_l`;
- outlier protection action `o_l`;
- residual correction rank `r_l`;
- KV-cache format `k_l` for attention state.

Let the resulting quantized operator be:

```
Q_l = Q(W_l; b_l, R_l, o_l, r_l)
```

The offline objective can be written as:

```
minimize    sum_l D_l(W_l, A_l, Q_l) + alpha * C_l(b_l, R_l, o_l, r_l, k_l)
subject to  sum_l M_l(b_l, k_l) <= B
            sum_l T_l(b_l, R_l, r_l, k_l) <= T
            Risk_l(o_l, b_l, k_l) <= epsilon_l
```

where:

- `D_l` is a distortion proxy such as activation MSE, output KL divergence, Hessian-weighted error, or Fisher-weighted error;
- `C_l` is implementation cost;
- `M_l` is memory;
- `T_l` is latency;
- `Risk_l` captures outlier or schema-safety risk.

The online variant introduces context `x_t`, which may include current sequence length, request type, recent KV-cache pressure, and device state. The policy is:

```
a_t = pi_theta(x_t),  a_t in A
```

where `A` contains bit allocation, rotation, residual, and KV-cache actions.

The constrained contextual bandit objective is:

```
maximize    E[Reward(x_t, a_t)]
subject to  E[Memory(x_t, a_t)] <= B
            E[Latency(x_t, a_t)] <= T
            P(QualityDrop(x_t, a_t) > delta) <= eta
```

A Lagrangian form is:

```
L(theta, lambda) =
  E[-Reward(x_t, pi_theta(x_t))]
  + lambda_M E[(Memory - B)_+]
  + lambda_T E[(Latency - T)_+]
  + lambda_Q E[(QualityDrop - delta)_+]
```

This formulation connects deterministic rules, supervised policy learning, and reinforcement learning. Deterministic bypass is the low-risk endpoint where `pi` is fixed and verifiable. Learned policies can be introduced only after their quality and safety can be measured.

4. EigenSkill-Q Design

4.1 Policy Skills

The current artifact defines five quantization-policy skills:

```
outlier_detect
bit_allocate
rotation_select
residual_patch
kv_policy
```

Each skill maps structured numerical evidence to a compact policy decision.

Example:

```
Input:
  Under budget medium, decide precision for layer_9 where sensitivity is
  1.828 and variance is 3.038.

Output:
  {"bits": 8, "format": "INT8", "score": 3.186, "skill": "bit_allocate"}
```

4.2 Semantic Router Versus Numeric Executor

The architecture separates two roles:

```
semantic router:
  identify whether the current request invokes a quantization-policy skill

policy kernel:
  compute the final numeric decision under audited rules
```

This prevents the model from becoming the sole executor of arithmetic thresholds and schema constraints. It also gives the system a unit-testable surface: the same dataset can evaluate Python, C++, or future ARM/NPU kernels.

4.3 Deterministic Bypass

For low-entropy policy functions, the bypass decision is:

```
if skill_id in deterministic_policy_set:
    return policy_kernel(skill_id, features)
else:
    return model.generate(prompt)
```

The important property is not that the policy is always hand-coded. The important property is that the executor is verifiable. A learned policy may be accepted into the bypass set only if it satisfies calibration and safety tests.

4.4 Learning Extension

After deterministic baselines are established, EigenSkill-Q can learn a policy over richer action spaces:

```
state:
  layer statistics, activation moments, Hessian/Fisher proxy, context length,
  memory pressure, device profile

action:
  bit width, format, rotation, residual rank, KV-cache policy

reward:
  - perplexity_delta - beta_1 memory - beta_2 latency - beta_3 parse_failure
```

A practical training route is:

1. Generate labels from strong offline quantization searches.
2. Train a small policy model by imitation.
3. Fine-tune with constrained bandit feedback on calibration workloads.
4. Export the policy into a deterministic or table-driven C++ executor when possible.

5. Current Artifact

Dataset

The committed v1 dataset contains:

```
train: 1200 rows
eval:   400 rows
test:   400 rows
```

It reports zero exact-row and zero input overlap across train/eval/test.

Python Oracle

The Python deterministic evaluator reaches:

split	rows	exact_json	decision_exact	parse_error
eval	400	1.000000	1.000000	0.000000
test	400	1.000000	1.000000	0.000000

C++ Policy Evaluator

The C++ evaluator reaches:

split	rows	policy_fields_exact	decision_exact	parse_error	throughput
eval	400	1.000000	1.000000	0.000000	8261 rows/s
test	400	1.000000	1.000000	0.000000	6924 rows/s

The C++ metric checks decision-bearing fields, not every auxiliary JSON field. Full JSON exactness is covered by the Python oracle.

Negative LoRA Evidence

A 60-sample SmolLM2-360M LoRA generation smoke test gives:

```
json_valid:     35%
schema_ok:       0%
exact_json:      0%
decision_exact:  0%
```

This is not a general model-quality claim. It is a local warning: short SFT on a tiny model is not a reliable executor for numeric quantization rules.

6. Experimental Plan For A Submission-Grade Paper

The current artifact is not enough for a CCF-A submission. The next experiments must convert the policy-layer idea into a real quantization evaluation.

Models

Minimum practical candidates:

- Qwen2.5-0.5B or Qwen2.5-1.5B;
- Llama-3.2-1B or Llama-3.2-3B if license and local access are convenient;
- SmolLM2-1.7B as a smaller open baseline.

Baselines

The paper must compare against:

- RTN or uniform quantization;
- GPTQ;
- AWQ;
- SmoothQuant;
- QuaRot or SpinQuant-style rotation;
- at least one KV-cache quantization baseline if long context is claimed.

Metrics

Minimum metrics:

- perplexity on WikiText/C4-style calibration and held-out data;
- downstream task accuracy on a small subset such as MMLU/GSM8K/BoolQ/ARC, depending on model size;
- memory footprint;
- policy-decision overhead;
- end-to-end latency;
- failure rate of router and policy schema.

Ablations

The decisive ablations are:

1. uniform precision versus sensitivity-guided bit allocation;
2. no rotation versus rotation policy;
3. no outlier protection versus outlier policy;
4. no residual patch versus residual-rank policy;
5. model-generated policy versus deterministic C++ policy kernel;
6. deterministic policy versus learned contextual bandit policy.

7. Venue Positioning

The user constraint is to avoid NeurIPS and MLSys. Under a CCF-A-oriented route, the current best targets are:

- **ICML**: if the constrained policy learning, rate-distortion theory, and quantization experiments become strong enough.
- **IJCAI / AAAI**: if the paper emphasizes AI policy learning plus robust empirical validation.
- **ACL**: if framed as reliable tool use / structured policy execution for language-model serving.
- **Artificial Intelligence, TPAMI, JMLR**: journal targets if experiments and theory become much deeper.

Strict note: TNNLS is a strong journal, but on the CCF AI official page it is listed as B, not A. It can be considered only if the user's school recognizes it through a separate journal list or CAS partition rule.

8. Limitations

The present artifact has several important limitations:

1. The dataset is synthetic.
2. The C++ evaluator is not integrated into a real LLM runtime.
3. The policies are threshold rules, not yet learned from real quantization searches.
4. No real model has been compressed by EigenSkill-Q yet.
5. No board-level edge latency or energy measurement exists.
6. No claim is made about true spectral/eigen-routing through nonlinear Transformer blocks.

These limitations should be stated directly. Overclaiming the current artifact would weaken the project.

9. Conclusion

EigenSkill-Q reframes part of LLM quantization as a verifiable policy-execution problem. The current evidence shows that deterministic quantization-policy skills can be represented as a no-leak dataset, exactly evaluated by a Python oracle, and executed by a standalone C++ policy kernel with perfect decision-field accuracy on the committed eval/test splits. The negative LoRA smoke test motivates this separation: small models are not reliable numeric policy executors under short training.

The next scientific step is clear: attach the policy layer to real mixed-precision quantization experiments, compare against standard baselines, and evaluate whether deterministic or learned policy bypass improves the quality-latency-memory tradeoff.

References And Source Links

- CCF Artificial Intelligence recommended publications: https://www.ccf.org.cn/Academic_Evaluation/AI
- GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers: https://arxiv.org/abs/2210.17323
- AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration: https://arxiv.org/abs/2306.00978
- SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models: https://arxiv.org/abs/2211.10438
- QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs: https://arxiv.org/abs/2404.00456
- SpinQuant: LLM Quantization with Learned Rotations: https://arxiv.org/abs/2405.16406
- QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks: https://arxiv.org/abs/2402.04396
- OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models: https://arxiv.org/abs/2308.13137
- AQLM: Extreme Compression of Large Language Models via Additive Quantization: https://arxiv.org/abs/2401.06118
- KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache: https://arxiv.org/abs/2402.02750
- KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization: https://arxiv.org/abs/2401.18079